Artificial Intelligence Blogs Posts

Connecting SAP GenAI Hub to n8n: A Complete Guide

Y

YangYue01

Product and Topic Expert

a month ago

5 Likes

1,495

SAP MANAGED TAGS

SAP AI Services | SAP AI Core | SAP AI Launchpad

A step-by-step tutorial on calling SAP GenAI Hub's large language models from self-hosted n8n to build enterprise AI automation workflows.

Top Liked Authors

Adeleruyelle 27

michael_ameling 25

shakhlo_ismailova57 11

chrwittm 10

vinayakgole 10

Luisen 7

SAP Build is the primary, recommended solution for enterprise automation and orchestration within the SAP ecosystem. However, one of the key strengths of **SAP GenAI Hub** is its standardization—it uses standard REST APIs and OAuth2, meaning it can be consumed by *any* system, regardless of the tech stack.

This tutorial demonstrates that openness. We will use **n8n** (a popular self-hosted workflow tool) to show how developers can integrate SAP's enterprise-grade GenAI capabilities into external or hybrid environments for rapid prototyping and testing.

Since n8n doesn't have a native SAP GenAI Hub node yet. No worries — we can easily make the connection using **HTTP Request + OAuth2**.

noravonthenen 7
A AndreasForster 5
S sherene_tan 5
R vodela 5
View all >

Prerequisites

Before we start, make sure you have:

1. **A running n8n instance** (Docker or npm or k8s)
2. **SAP BTP account** with SAP AI Core service enabled (Extended Plan for GenAI Hub)
3. **A deployed LLM model** (configured in SAP AI Launchpad)

Get Your SAP AI Core Service Key

Navigate to SAP BTP Cockpit → Your Subaccount → Instances and Subscriptions → SAP AI Core → Create Service Key

You'll get a JSON like this:

```
{
  "clientid": "sb-xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx!b12345|
  "clientsecret": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx$xxxxxx
  "url": "https://your-tenant.authentication.eu10.hana.ondeman
  "serviceurls": {
    "AI_API_URL": "https://api.ai.prod.eu-central-1.aws.ml.han
  }
}
```

Keep these four fields handy — you'll need them all.

Get Your Deployment ID

In SAP AI Launchpad:

1. Go to **ML Operations** → **Deployments**
2. Find your deployed model
3. Note the `Deployment ID` (or the full `deploymentUrl`)

Step 1: Create OAuth2 Credential

SAP GenAI Hub uses OAuth2 Client Credentials authentication. Let's configure the credential in n8n first.

1. Open n8n, go to **Settings** → **Credentials**
2. Click **Add Credential**, search for **OAuth2 API**
3. Fill in the following:

Field	Value
Credential Name	`SAP-GenAI-Hub`
Grant Type	`Client Credentials`
Access Token URL	`https://your-tenant.authentication.eu10.hana.ondemand.com/oauth/to`
Client ID	Copy the full `clientid` from Service Key
Client Secret	Copy the full `clientsecret` from Service Key
Scope	Leave empty
Authentication	**Select** `Header`

4. Click **Save**

> 💡 **Common Mistake**: Selecting "Send as Basic Auth Header" will cause 401 authentication failures. SAP requires credentials to be sent in the body.

Step 2: Build a Simple Q&A Workflow

Let's create a basic LLM query workflow.

2.1 Add Manual Trigger

Start with a manual trigger for testing:

- Add a **Manual Trigger** node

2.2 Add HTTP Request Node

This is the core node. Configure as follows:

Basic Settings:

- **Method**: `POST`
- **URL**:

```
https://api.ai.prod.eu-central-1.aws.ml.hana.ondemand.com/v2/i
```

⚠️ Note: If using OpenAI models, you should add the api-version parameter, e.g.

```
https://api.ai.prod.eu-central-1.aws.ml.hana.ondemand.com/v2/i
```

Authentication:

- **Authentication**: `Predefined Credential Type`
- **Credential Type**: `OAuth2 API`
- **OAuth2 API**: Select `SAP-GenAI-Hub`

Headers:

Name	Value
`Content-Type`	`application/json`
`AI-Resource-Group`	`default` , or your own Resource Group ID

Body:

- **Body Content Type**: `JSON`
- **Specify Body**: `Using JSON`

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are a professional assistant. Please pro
    },
    {
      "role": "user",
      "content": "What is SAP S/4HANA? Please summarize in thr
    }
  ]
}
```

2.3 Test It

Click **Test Workflow**. If configured correctly, you'll see a response like:

```
{
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "SAP S/4HANA is SAP's next-generation intel
      }
    }
  ],
  "usage": {
    "prompt_tokens": 45,
    "completion_tokens": 128,
    "total_tokens": 173
  }
}
```


🎉**Congratulations! You've successfully called SAP GenAI Hub from n8n!**

Step 3: Build an Interactive Chat Workflow

Let's upgrade to a conversational version.

Workflow Structure

Chat Trigger → HTTP Request (SAP GenAI Hub) → Code (Extract Reply)

3.1 Chat Trigger Node

Add a **Chat Trigger** node (found under Advanced AI category):

- Keep default settings
- It provides an embedded chat interface for testing

3.2 Modify HTTP Request Node

Change the Body to dynamically get user input:

```
{
  "messages": [
    {
      "role": "system",
      "content": "You are a professional enterprise consultant
    },
    {
      "role": "user",
      "content": "{{ $json.chatInput }}"
    }
  ]
}
```

`{{ $json.chatInput }}` automatically retrieves the user's input from the chat interface.

3.3 Add Code Node to Extract Response

Add a **Code** node to extract the AI's reply:

```
const response = $input.all()[0].json;
const aiMessage = response.choices?.[0]?.message?.content || "

return [{
  json: {
    response: aiMessage  }
}];
```

3.4 Test the Chat

1. Activate the Workflow
2. Click the **Chat** button to open the chat interface
3. Start chatting!

Step 4: Add Conversation Memory (Optional)

The above version treats each message independently without context. We can use n8n's **Workflow Static Data** to implement simple memory.

Improved Workflow

Add a **Code** node before HTTP Request to manage conversation history:

```
// Get session ID and user input
const sessionId = $json.sessionId || 'default';
const userInput = $json.chatInput;

// Get history from global static data
const staticData = $getWorkflowStaticData('global');
if (!staticData.conversations) {
  staticData.conversations = {};
}
if (!staticData.conversations[sessionId]) {
  staticData.conversations[sessionId] = [
    {
      role: "system",
      content: "You are a professional enterprise consultant s
    }
  ];
}

// Add user message
staticData.conversations[sessionId].push({
  role: "user",
  content: userInput});

// Keep only the last 20 messages (avoid token limit)
const messages = staticData.conversations[sessionId].slice(-20

return [{
  json: {
    sessionId,
```

```
    messages  }
}];
```

Then modify the HTTP Request Body:

```
{
  "messages": {{ JSON.stringify($json.messages) }}
}
```

Finally, save the assistant's reply in the response extraction Code node:

```
const response = $input.all()[0].json;
const sessionId = $('Prepare Messages').first().json.sessionId
const aiMessage = response.choices?.[0]?.message?.content || "

// Save assistant reply to history
const staticData = $getWorkflowStaticData('global');
staticData.conversations[sessionId].push({
  role: "assistant",
  content: aiMessage});

return [{
  json: {
    response: aiMessage  }
}];
```

Now your chatbot remembers context!

Practical Use Cases

Use Case 1: Automated Email Replies

Email Trigger → Extract Content → SAP GenAI Hub → Send Reply

Sample Prompt:

```
Based on the following customer email, generate a professional
Email content:{{ $json.emailBody }}
Requirements:1. Professional and friendly tone2. If it's a pri
```

Use Case 2: Document Summarization

Webhook → Read PDF → SAP GenAI Hub → Save to Notion

Use Case 3: Data Analysis Assistant

Combine SAP APIs to fetch business data and let AI generate analysis reports:

Schedule Trigger → SAP S/4HANA → SAP GenAI Hub → Slack Message

Troubleshooting

❌401 Unauthorized

Cause: Authentication failed

Check:

- Client ID and Secret are fully copied (including special characters like `!`, `|`, `$`)
- Authentication is set to `Send In Body`
- Token URL format is correct (ends with `/oauth/token`)

❌404 Not Found

Cause: API path error

Check:

- Deployment ID is correct
- Model is successfully deployed (status shows "Running" in AI Launchpad)
- API URL path format: `/v2/inference/deployments/{id}/chat/completions`

❌403 Forbidden

Cause: Insufficient permissions

Check:

- `AI-Resource-Group` header is set (default is `default`)

- User has permission to access the deployment

❌400 Bad Request

Cause: Invalid request body

Check:

- JSON format is correct
- `messages` is an array
- Each message has both `role` and `content`

Optimization Tips

1. **Limit conversation history**: Keep only the last N turns to avoid exceeding context window
2. **Add error handling**: Use n8n's Error Trigger to handle API failures gracefully
3. **Use variables**: Store API URLs and Deployment IDs in n8n Variables for easier management

---

Summary

In this tutorial, you learned:

✅How to configure OAuth2 authentication for SAP GenAI Hub in n8n
✅How to use HTTP Request node to call LLM APIs
✅How to build a chatbot with conversation memory

✔️How to troubleshoot common errors

The power of n8n lies in its flexibility — you can seamlessly integrate SAP GenAI Hub with hundreds of other applications (Email, Databases, SAP S/4HANA, etc.) to build truly enterprise-grade AI automation workflows.

Happy Automating! 🚀

LABELS

API Integration | GenAI Assisted Content | GenAI Hub | LLM | n8n | SAP AI Core | Workflow Automation

TAGS

GENERATIVE AI HUB | Integration | LLM | Low-Code | N8n | Oauth2 | REST API | Sap Ai Core | Sap Btp | Workflow Automation

1 Comment

K

KishoreMarodia
Explorer

a month ago

0 Likes

Thanks for shining light on n8n integration with SAP GEN AI, this surely opens up lots of use cases.

You must be a registered user to add a comment. If you've already registered, sign in. Otherwise, register and sign in.

Comment

Follow

Privacy

Terms of Use

Copyright

Legal Disclosure

Trademark

Support

Cookie Preferences